

Algoritmos e Estruturas de Dados

Fundamentos Essenciais para Desenvolvedores

DeepStudy PDF Generator

July 31, 2026

Contents

1	Fundamentos e Teoria Essencial	3
1.1	Por que estruturas de dados e algoritmos importam	3
1.2	Notação assintótica: O , Ω e Θ	3
1.3	Complexidade no melhor, médio e pior caso	4
2	Estruturas Lineares	6
2.1	Arrays	6
2.2	Listas Ligadas	7
2.3	Pilhas (Stacks)	9
2.4	Filas (Queues)	10
3	Árvores	12
3.1	Árvores Binárias e Árvores de Busca Binária (BST)	12
3.2	Árvores AVL	13
3.3	Árvores Rubro-Negras	15
3.4	B-Trees	17
4	Grafos	20
4.1	Representação de Grafos	20
4.2	Busca em Largura (BFS)	21
4.3	Busca em Profundidade (DFS)	22
4.4	Algoritmo de Dijkstra	23
5	Algoritmos de Ordenação	25
5.1	Bubble Sort	25

5.2	Selection Sort	26
5.3	Insertion Sort	26
5.4	Merge Sort	27
5.5	Quick Sort	28
5.6	Heap Sort	29
5.7	Comparativo dos Algoritmos de Ordenação	30
6	Tabelas Hash e Dicionários	31
6.1	Hashing e Resolução de Colisões	31
7	Heaps e Filas de Prioridade	33
7.1	Binary Heap	33
8	Técnicas de Projeto de Algoritmos	36
8.1	Divisão e Conquista	36
8.2	Programação Dinâmica	37
8.3	Algoritmos Gulosos (Greedy)	39
A	Referências e Leitura Adicional	41
B	Glossário	42
	Index	43

Chapter 1

Fundamentos e Teoria Essencial

1.1 Por que estruturas de dados e algoritmos importam

Todo programa é, no fundo, a combinação de duas coisas: **dados** organizados de alguma forma e **operações** que transformam esses dados. A escolha da estrutura de dados certa e do algoritmo certo é o que separa um sistema que atende 100 usuários de um que atende 100 milhões com o mesmo hardware. Um exemplo clássico: buscar um elemento em uma lista não ordenada de um milhão de itens pode levar um milhão de comparações; a mesma busca em uma árvore balanceada ou tabela hash leva, na prática, menos de 20 ou até apenas 1 comparação. Essa diferença não é otimização de detalhe — é a diferença entre um produto viável e um que trava.

Este livro cobre as estruturas e algoritmos que aparecem, direta ou indiretamente, em praticamente todo sistema de software não-trivial: estruturas lineares (arrays, listas ligadas, pilhas, filas), árvores (binárias, AVL, rubro-negras, B-trees), grafos (com BFS, DFS e Dijkstra), algoritmos de ordenação, tabelas hash e heaps, além de três técnicas de projeto de algoritmos que atravessam todos esses temas: divisão e conquista, programação dinâmica e algoritmos gulosos.

1.2 Notação assintótica: O , Ω e Θ

Para comparar algoritmos de forma independente da máquina, do compilador ou da linguagem, usamos **notação assintótica**. Ela descreve como o tempo de execução (ou o consumo de memória) cresce à medida que o tamanho da entrada n cresce, ignorando constantes e termos de baixa ordem.

Big-O (O) — limite superior. $f(n) = O(g(n))$ significa que, a partir de um certo ponto, $f(n)$ nunca cresce mais rápido que $g(n)$ multiplicado por uma constante. Formalmente, existem constantes $c > 0$ e n_0 tais que $f(n) \leq c \cdot g(n)$ para todo $n \geq n_0$. Big-O descreve o *pior caso* — uma garantia de que o algoritmo nunca será pior que aquilo.

Big-Omega (Ω) — limite inferior. $f(n) = \Omega(g(n))$ significa o oposto: $f(n)$ nunca cresce mais devagar que $g(n)$ (a menos de constante). Existem $c > 0$ e n_0 tais que

$f(n) \geq c \cdot g(n)$ para todo $n \geq n_0$. Descreve o *melhor caso* possível de um algoritmo.

Big-Theta (Θ) — limite justo. $f(n) = \Theta(g(n))$ ocorre quando $f(n) = O(g(n))$ e $f(n) = \Omega(g(n))$ simultaneamente — ou seja, $g(n)$ é ao mesmo tempo cota superior e inferior, a menos de constantes. Quando dizemos que um algoritmo “roda em tempo $\Theta(n \log n)$ ”, estamos afirmando que ele nunca é mais rápido nem mais lento que isso, para qualquer entrada de tamanho n .

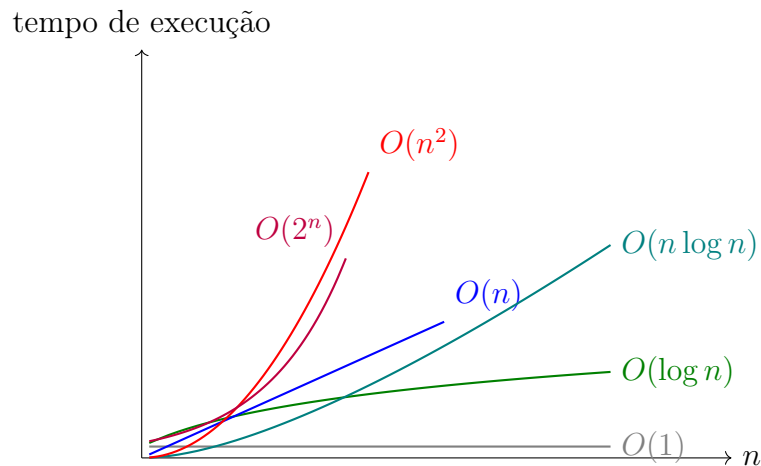


Figure 1.1: Crescimento comparativo das classes de complexidade mais comuns. Note como $O(2^n)$ sai do gráfico rapidamente — é por isso que algoritmos exponenciais são inviáveis para entradas grandes.

Análise de Complexidade

Regras práticas para derivar complexidade:

- Loops aninhados multiplicam: dois loops de n dentro um do outro $\Rightarrow O(n^2)$.
- Loops sequenciais somam, e a soma vira o maior termo: um loop $O(n)$ seguido de outro $O(n^2) \Rightarrow O(n^2)$.
- Dividir o problema pela metade a cada passo introduz um fator $\log n$ (base 2, mas a base não importa em notação O).
- Constantes e termos de baixa ordem são descartados: $3n^2 + 100n + 5$ é $O(n^2)$.
- Complexidade de espaço segue as mesmas regras, mas mede memória extra alocada, não tempo.

1.3 Complexidade no melhor, médio e pior caso

Muitos algoritmos têm comportamento diferente dependendo da entrada. Quicksort, por exemplo, é $\Theta(n \log n)$ no caso médio mas $O(n^2)$ no pior caso (quando o pivô escolhido é sempre o menor ou maior elemento). Ao longo deste livro, cada estrutura e algoritmo traz uma tabela de complexidade explicitando melhor, médio e pior caso sempre que eles

divergem — é um erro comum (e perigoso em produção) assumir que a complexidade média sempre se aplica.

Chapter 2

Estruturas Lineares

2.1 Arrays

Um **array** é um bloco contíguo de memória onde cada elemento ocupa um espaço de tamanho fixo, permitindo calcular o endereço de qualquer posição i diretamente: $\text{endereço} = \text{base} + i \times \text{tamanho do elemento}$. É essa aritmética de endereço que torna o acesso por índice $O(1)$.

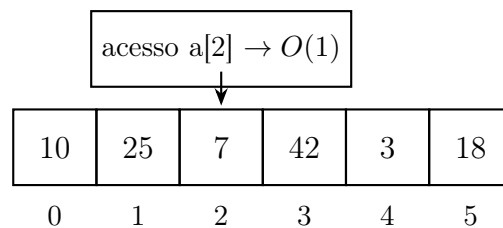


Figure 2.1: Array contíguo: acesso direto pelo índice é aritmética de ponteiro, portanto $O(1)$. Inserir no meio exige deslocar todos os elementos à direita.

```
1 class DynamicArray<T> {
2   private data: T[] = [];
3
4   get(index: number): T {
5     if (index < 0 || index >= this.data.length) {
6       throw new RangeError("index out of bounds");
7     }
8     return this.data[index]; // O(1) - aritmética de endereço
9   }
10
11   set(index: number, value: T): void {
12     this.data[index] = value; // O(1)
13   }
14
15   push(value: T): void {
16     this.data.push(value); // O(1) amortizado
```

```

17   }
18
19   insertAt(index: number, value: T): void {
20     this.data.splice(index, 0, value); // O(n) - desloca elementos
21   }
22
23   removeAt(index: number): T {
24     return this.data.splice(index, 1)[0]; // O(n) - desloca elementos
25   }
26
27   get length(): number {
28     return this.data.length;
29   }
30 }

```

Análise de Complexidade

Operação	Melhor	Médio	Pior
Acesso por índice	$O(1)$	$O(1)$	$O(1)$
Busca (valor)	$O(1)$	$O(n)$	$O(n)$
Inserção no fim	$O(1)$	$O(1)$ amort.	$O(n)$ (realloc)
Inserção no meio/início	$O(n)$	$O(n)$	$O(n)$
Remoção no meio/início	$O(n)$	$O(n)$	$O(n)$

2.2 Listas Ligadas

Uma **lista ligada** troca a contiguidade do array por flexibilidade: cada elemento (nó) guarda seu valor e um ponteiro para o próximo nó. Isso torna inserção e remoção nas extremidades $O(1)$, mas elimina o acesso direto por índice — para chegar ao elemento i é preciso percorrer i nós a partir da cabeça.

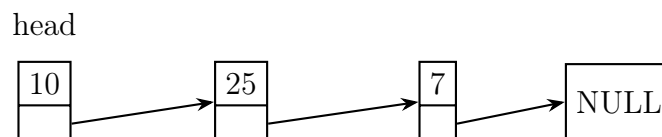


Figure 2.2: Lista ligada simples (*singly linked list*): cada nó aponta apenas para o próximo. Listas duplamente ligadas adicionam um ponteiro `prev` para navegação nos dois sentidos.

```

1  class ListNode<T> {
2    value: T;
3    next: ListNode<T> | null = null;
4    constructor(value: T) {
5      this.value = value;
6    }

```

```

7  }
8
9  class LinkedList<T> {
10     private head: ListNode<T> | null = null;
11     private tail: ListNode<T> | null = null;
12     private size = 0;
13
14     pushFront(value: T): void {
15         const node = new ListNode(value);
16         node.next = this.head;
17         this.head = node;
18         if (!this.tail) this.tail = node;
19         this.size++; // O(1)
20     }
21
22     pushBack(value: T): void {
23         const node = new ListNode(value);
24         if (this.tail) this.tail.next = node;
25         else this.head = node;
26         this.tail = node;
27         this.size++; // O(1) graças ao ponteiro tail
28     }
29
30     find(value: T): ListNode<T> | null {
31         let current = this.head;
32         while (current) {
33             if (current.value === value) return current; // O(n)
34             current = current.next;
35         }
36         return null;
37     }
38
39     removeFront(): T | null {
40         if (!this.head) return null;
41         const value = this.head.value;
42         this.head = this.head.next;
43         if (!this.head) this.tail = null;
44         this.size--;
45         return value; // O(1)
46     }
47 }

```

Análise de Complexidade

Operação	Lista simples	Lista dupla	Array
Inserção no início	$O(1)$	$O(1)$	$O(n)$
Inserção no fim	$O(1)^*$	$O(1)$	$O(1)$ amort.
Acesso por índice	$O(n)$	$O(n)$	$O(1)$
Remoção dado o nó	$O(n)$	$O(1)$	$O(n)$

* com ponteiro tail mantido.

2.3 Pilhas (Stacks)

Uma **pilha** segue a disciplina LIFO (*Last In, First Out*): o último elemento inserido é o primeiro a sair. É a estrutura por trás da recursão (pilha de chamadas), do desfazer/refazer de editores, da avaliação de expressões e da verificação de parênteses balanceados.

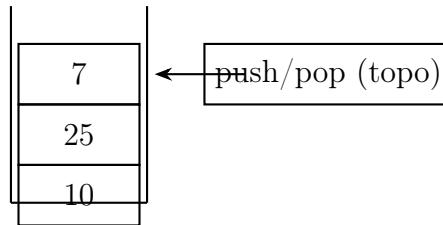


Figure 2.3: Pilha: inserção e remoção ocorrem sempre no topo, ambas $O(1)$.

```

1  class Stack<T> {
2      private items: T[] = [];
3
4      push(value: T): void {
5          this.items.push(value); // O(1) amortizado
6      }
7
8      pop(): T | undefined {
9          return this.items.pop(); // O(1)
10     }
11
12     peek(): T | undefined {
13         return this.items[this.items.length - 1]; // O(1)
14     }
15
16     isEmpty(): boolean {
17         return this.items.length === 0;
18     }
19 }
20
21 // Exemplo clássico: validar parênteses balanceados
22 function isBalanced(expr: string): boolean {
23     const stack = new Stack<string>();
24     const pairs: Record<string, string> = { ")": "(", "]" : "[" , "}": "{" };
25     for (const ch of expr) {
26         if ("([[".includes(ch)) stack.push(ch);
27         else if (ch in pairs) {
28             if (stack.isEmpty() || stack.pop() !== pairs[ch]) return false;
29         }
30     }
31     return stack.isEmpty(); // O(n) no total
32 }

```

Análise de Complexidade

Push, pop e peek são todos $O(1)$ (implementados sobre array dinâmico ou lista ligada). Espaço: $O(n)$ para n elementos armazenados.

2.4 Filas (Queues)

Uma **fila** segue a disciplina FIFO (*First In, First Out*): o primeiro elemento a entrar é o primeiro a sair. É a base de filas de impressão, filas de mensagens (RabbitMQ, Kafka), escalonamento de processos e, mais adiante neste livro, do algoritmo BFS em grafos.

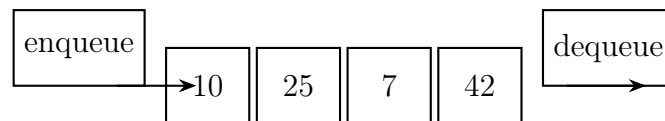


Figure 2.4: Fila: inserção (*enqueue*) sempre no fim, remoção (*dequeue*) sempre no início.

```

1  // Implementação com lista ligada evita o custo  $O(n)$  de shift()
2  // que um array puro teria ao remover do início.
3  class QueueNode<T> {
4      constructor(public value: T, public next: QueueNode<T> | null = null) {}
5  }
6
7  class Queue<T> {
8      private head: QueueNode<T> | null = null;
9      private tail: QueueNode<T> | null = null;
10     private count = 0;
11
12     enqueue(value: T): void {
13         const node = new QueueNode(value);
14         if (this.tail) this.tail.next = node;
15         else this.head = node;
16         this.tail = node;
17         this.count++; //  $O(1)$ 
18     }
19
20     dequeue(): T | undefined {
21         if (!this.head) return undefined;
22         const value = this.head.value;
23         this.head = this.head.next;
24         if (!this.head) this.tail = null;
25         this.count--;
26         return value; //  $O(1)$ 
27     }
28
29     get size(): number {
30         return this.count;
31     }

```

}

Análise de Complexidade

Enqueue e dequeue são $O(1)$ quando implementados com lista ligada (ponteiros head/tail) ou com buffer circular sobre array. Um array comum com `shift()` custa $O(n)$ por remoção, pois desloca todos os elementos — armadilha comum em código JavaScript/TypeScript iniciante.

Chapter 3

Árvores

3.1 Árvores Binárias e Árvores de Busca Binária (BST)

Uma **árvore binária** é uma estrutura hierárquica onde cada nó tem no máximo dois filhos (esquerdo e direito). Uma **árvore de busca binária** (BST) impõe uma invariante: para todo nó, todos os valores da subárvore esquerda são menores, e todos os da subárvore direita são maiores. Essa invariante é o que torna busca, inserção e remoção $O(\log n)$ — desde que a árvore esteja balanceada.

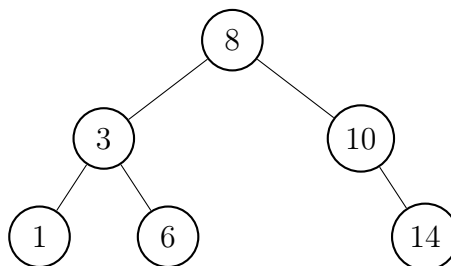


Figure 3.1: BST: para o nó 8, tudo à esquerda (≤ 6) é menor, tudo à direita (≥ 10) é maior. Busca por 6: $8 \rightarrow 3 \rightarrow 6$, três comparações em vez de percorrer todos os nós.

```
1 class TreeNode<T> {
2   left: TreeNode<T> | null = null;
3   right: TreeNode<T> | null = null;
4   constructor(public value: T) {}
5 }
6
7 class BST<T> {
8   private root: TreeNode<T> | null = null;
9
10  insert(value: T): void {
11    const node = new TreeNode(value);
12    if (!this.root) { this.root = node; return; }
13    let current = this.root;
14    while (true) {
```

```

15     if (value < current.value) {
16         if (!current.left) { current.left = node; return; }
17         current = current.left;
18     } else {
19         if (!current.right) { current.right = node; return; }
20         current = current.right;
21     }
22 } // O(h), h = altura da árvore
23
24
25 search(value: T): boolean {
26     let current = this.root;
27     while (current) {
28         if (value === current.value) return true;
29         current = value < current.value ? current.left : current.right;
30     }
31     return false; // O(h)
32 }
33
34 inOrder(): T[] {
35     const result: T[] = [];
36     const visit = (node: TreeNode<T> | null) => {
37         if (!node) return;
38         visit(node.left);
39         result.push(node.value);
40         visit(node.right);
41     };
42     visit(this.root);
43     return result; // O(n) - percorre todos os nós, produz saída ordenada
44 }
45 }

```

Análise de Complexidade

Operação	Balanceada	Degenerada (lista)
Busca	$O(\log n)$	$O(n)$
Inserção	$O(\log n)$	$O(n)$
Remoção	$O(\log n)$	$O(n)$

3.2 Árvores AVL

Uma BST comum pode degenerar em uma lista ligada se os valores forem inseridos em ordem (ex.: 1, 2, 3, 4, 5), destruindo a garantia de $O(\log n)$. A **árvore AVL** resolve isso mantendo, para todo nó, a diferença de altura entre as subárvores esquerda e direita (*fator de balanceamento*) sempre em $\{-1, 0, 1\}$. Quando uma inserção ou remoção viola essa invariante, rotações restauram o balanceamento.

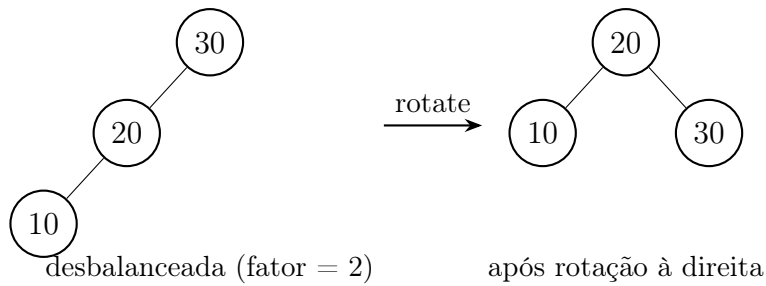


Figure 3.2: Rotação simples à direita corrigindo o caso “esquerda-esquerda”. Existem quatro casos: esquerda-esquerda, direita-direita, esquerda-direita e direita-esquerda, os dois últimos exigindo uma rotação dupla.

```

1  class AVLNode<T> {
2      left: AVLNode<T> | null = null;
3      right: AVLNode<T> | null = null;
4      height = 1;
5      constructor(public value: T) {}
6  }
7
8  function height(node: AVLNode<any> | null): number {
9      return node ? node.height : 0;
10 }
11
12 function balanceFactor(node: AVLNode<any>): number {
13     return height(node.left) - height(node.right);
14 }
15
16 function updateHeight(node: AVLNode<any>): void {
17     node.height = 1 + Math.max(height(node.left), height(node.right));
18 }
19
20 function rotateRight<T>(y: AVLNode<T>): AVLNode<T> {
21     const x = y.left!;
22     y.left = x.right;
23     x.right = y;
24     updateHeight(y);
25     updateHeight(x);
26     return x; // nova raiz da subárvore
27 }
28
29 function rotateLeft<T>(x: AVLNode<T>): AVLNode<T> {
30     const y = x.right!;
31     x.right = y.left;
32     y.left = x;
33     updateHeight(x);
34     updateHeight(y);
35     return y;
36 }
37
38 function insertAVL<T>(node: AVLNode<T> | null, value: T): AVLNode<T> {

```

```

39  if (!node) return new AVLNode(value);
40  if (value < node.value) node.left = insertAVL(node.left, value);
41  else node.right = insertAVL(node.right, value);
42
43  updateHeight(node);
44  const balance = balanceFactor(node);
45
46  if (balance > 1 && value < node.left!.value) return rotateRight(node);
47  if (balance < -1 && value > node.right!.value) return rotateLeft(node);
48  if (balance > 1 && value > node.left!.value) {
49      node.left = rotateLeft(node.left!);
50      return rotateRight(node);
51  }
52  if (balance < -1 && value < node.right!.value) {
53      node.right = rotateRight(node.right!);
54      return rotateLeft(node);
55  }
56  return node; // O(log n) - altura garantida por invariante
57  }

```

Análise de Complexidade

Busca, inserção e remoção são **sempre** $O(\log n)$, mesmo no pior caso — essa é exatamente a garantia que uma BST comum não oferece. O custo extra é o rebalanceamento a cada inserção/remoção, que ainda é $O(\log n)$ (no máximo $O(\log n)$ rotações no caminho até a raiz).

3.3 Árvores Rubro-Negras

Árvores **rubro-negras** (red-black trees) são outra forma de BST auto-balanceada, usada internamente em `TreeMap/TreeSet` do Java, `std::map` do C++ e no escalonador CFS do kernel Linux. Em vez de manter altura estritamente balanceada como a AVL, elas garantem apenas que nenhum caminho da raiz até uma folha seja mais que o dobro de outro, via 5 invariantes de coloração:

1. Todo nó é vermelho ou preto.
2. A raiz é sempre preta.
3. Toda folha nula (NIL) é preta.
4. Um nó vermelho nunca tem filho vermelho (sem dois vermelhos seguidos).
5. Todo caminho da raiz a uma folha NIL tem o mesmo número de nós pretos (“altura preta”).

Essas regras são mais frouxas que as da AVL, o que significa **menos rotações** por inserção (bom para escritas frequentes), ao custo de buscas ligeiramente menos rápidas na prática — é um trade-off clássico leitura-vs-escrita.

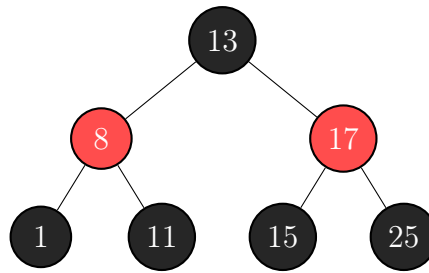


Figure 3.3: Árvore rubro-negra válida: raiz preta, nenhum vermelho com filho vermelho, e toda folha NIL (omitida) vê o mesmo número de nós pretos no caminho.

```

1  enum Color { RED, BLACK }
2
3  class RBNode<T> {
4      color: Color = Color.RED; // novo nó sempre entra vermelho
5      left: RBNode<T> | null = null;
6      right: RBNode<T> | null = null;
7      parent: RBNode<T> | null = null;
8      constructor(public value: T) {}
9  }
10
11  // Após inserir como BST comum e colorir de vermelho, o "fixup"
12  // resolve violações subindo pela árvore, usando o "tio" do nó
13  // para decidir entre recolorir ou rotacionar.
14  function insertFixup<T>(tree: RedBlackTree<T>, node: RBNode<T>): void {
15      while (node.parent && node.parent.color === Color.RED) {
16          const grandparent = node.parent.parent!;
17          const isParentLeft = node.parent === grandparent.left;
18          const uncle = isParentLeft ? grandparent.right : grandparent.left;
19
20          if (uncle && uncle.color === Color.RED) {
21              // Caso 1: tio vermelho -> apenas recolorir e subir
22              node.parent.color = Color.BLACK;
23              uncle.color = Color.BLACK;
24              grandparent.color = Color.RED;
25              node = grandparent;
26          } else {
27              // Caso 2/3: tio preto -> rotação(ões) + recoloração
28              if (isParentLeft && node === node.parent.right) {
29                  node = node.parent;
30                  tree.rotateLeft(node);
31              }
32              node.parent!.color = Color.BLACK;
33              grandparent.color = Color.RED;
34              isParentLeft ? tree.rotateRight(grandparent) :
35                          tree.rotateLeft(grandparent);
36          }
37      }
38      tree.root!.color = Color.BLACK; // invariante 2
39  }

```

Análise de Complexidade

Busca, inserção e remoção: $O(\log n)$ garantido no pior caso, pois a altura da árvore nunca excede $2\log_2(n + 1)$. Comparada à AVL: mais rápida em cargas de escrita intensa (menos rotações por inserção), levemente mais lenta em buscas puras (árvore pode ficar até 2x mais alta que uma AVL equivalente).

3.4 B-Trees

Enquanto AVL e rubro-negra assumem acesso à memória RAM (onde qualquer ponteiro custa o mesmo), **B-trees** foram desenhadas para minimizar acessos a **disco**, onde cada leitura é ordens de magnitude mais cara. A ideia: cada nó guarda *múltiplas chaves* (não apenas uma) e *múltiplos filhos*, reduzindo drasticamente a altura da árvore. É a estrutura por trás de índices em bancos de dados relacionais (PostgreSQL, MySQL/InnoDB) e sistemas de arquivos.

Uma B-tree de ordem m satisfaz: cada nó tem entre $\lceil m/2 \rceil - 1$ e $m - 1$ chaves (exceto a raiz), as chaves dentro de um nó ficam ordenadas, e um nó com k chaves tem exatamente $k + 1$ filhos — cada filho contendo valores entre duas chaves consecutivas do pai.

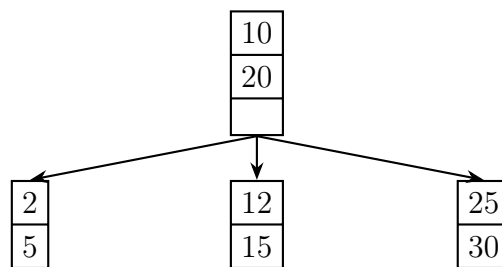


Figure 3.4: B-tree de ordem 3: a raiz com chaves $[10, 20]$ tem três filhos — valores < 10 , entre 10 e 20, e > 20 . Cada nó mapeia para uma página de disco; poucos níveis cobrem milhões de registros.

```

1  class BTreeNode<T> {
2      keys: T[] = [];
3      children: BTreeNode<T>[] = [];
4      leaf = true;
5      constructor(private minDegree: number) {}
6
7      get isFull(): boolean {
8          return this.keys.length === 2 * this.minDegree - 1;
9      }
10 }
11
12 class BTree<T> {
13     private root: BTreeNode<T>;
14     constructor(private t: number) { // t = grau mínimo
15         this.root = new BTreeNode(t);
  
```

```

16     }
17
18     search(node: BTreeNode<T>, key: T): boolean {
19         let i = 0;
20         while (i < node.keys.length && key > node.keys[i]) i++;
21         if (i < node.keys.length && key === node.keys[i]) return true;
22         if (node.leaf) return false;
23         return this.search(node.children[i], key); // O(log n)
24     }
25
26     // Inserção: se a raiz está cheia, cria nova raiz e divide (split)
27     // o filho cheio ANTES de descer - garante espaço para inserir
28     // sem nunca precisar "voltar" na recursão.
29     insert(key: T): void {
30         if (this.root.isFull) {
31             const newRoot = new BTreeNode<T>(this.t);
32             newRoot.leaf = false;
33             newRoot.children.push(this.root);
34             this.splitChild(newRoot, 0);
35             this.root = newRoot;
36         }
37         this.insertNonFull(this.root, key);
38     }
39
40     private splitChild(parent: BTreeNode<T>, index: number): void {
41         const t = this.t;
42         const full = parent.children[index];
43         const sibling = new BTreeNode<T>(t);
44         sibling.leaf = full.leaf;
45         sibling.keys = full.keys.splice(t, t - 1);
46         if (!full.leaf) sibling.children = full.children.splice(t, t);
47         parent.keys.splice(index, 0, full.keys.splice(t - 1, 1)[0]);
48         parent.children.splice(index + 1, 0, sibling);
49     }
50
51     private insertNonFull(node: BTreeNode<T>, key: T): void {
52         let i = node.keys.length - 1;
53         if (node.leaf) {
54             node.keys.push(key as any);
55             while (i >= 0 && key < node.keys[i]) {
56                 node.keys[i + 1] = node.keys[i];
57                 i--;
58             }
59             node.keys[i + 1] = key;
60         } else {
61             while (i >= 0 && key < node.keys[i]) i--;
62             i++;
63             if (node.children[i].isFull) {
64                 this.splitChild(node, i);
65                 if (key > node.keys[i]) i++;
66             }
67             this.insertNonFull(node.children[i], key);

```

```
68     }  
69   }  
70 }
```

Análise de Complexidade

Busca, inserção e remoção: $O(\log n)$, mas com uma **base de log muito maior** que 2 (tipicamente centenas), pois cada nó guarda muitas chaves. Isso significa uma árvore de altura 3–4 para milhões de registros — 3–4 acessos a disco, contra dezenas em uma AVL/rubro-negra equivalente. É por isso que bancos de dados usam B-trees (ou variantes B+tree) para índices, não árvores binárias simples.

Chapter 4

Grafos

4.1 Representação de Grafos

Um **grafo** $G = (V, E)$ é um conjunto de vértices V e arestas E conectando pares de vértices. Grafos modelam redes sociais, mapas de rotas, dependências de pacotes, links entre páginas web — qualquer relação “ponto a ponto”. As duas representações mais comuns são **lista de adjacência** e **matriz de adjacência**, com trade-offs opostos de espaço e velocidade.

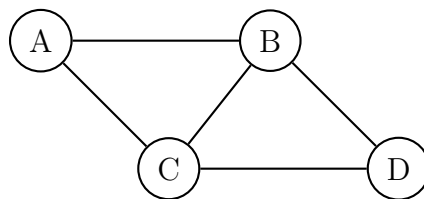


Figure 4.1: Grafo não-dirigido com 4 vértices e 5 arestas, usado nos exemplos de BFS e DFS a seguir.

```
1 class Graph {
2     private adjacency = new Map<string, string[]>();
3
4     addVertex(v: string): void {
5         if (!this.adjacency.has(v)) this.adjacency.set(v, []);
6     }
7
8     addEdge(u: string, v: string): void {
9         this.addVertex(u);
10        this.addVertex(v);
11        this.adjacency.get(u)!.push(v);
12        this.adjacency.get(v)!.push(u); // não-dirigido
13    }
14
15    neighbors(v: string): string[] {
16        return this.adjacency.get(v) ?? []; // O(1) para obter a lista
```

```

17 }
18 }

```

Análise de Complexidade

Operação	Lista de adjacência	Matriz de adjacência
Espaço	$O(V + E)$	$O(V^2)$
Verificar aresta (u, v)	$O(\deg(u))$	$O(1)$
Listar vizinhos de v	$O(\deg(v))$	$O(V)$

Lista de adjacência é a escolha padrão para grafos esparsos ($E \ll V^2$, o caso mais comum); matriz vale a pena só para grafos densos ou quando checagem de aresta $O(1)$ é crítica.

4.2 Busca em Largura (BFS)

BFS (*Breadth-First Search*) explora o grafo em “camadas”: visita todos os vizinhos diretos do vértice inicial antes de avançar para os vizinhos dos vizinhos. Usa uma **fila** (a estrutura FIFO do Capítulo 2) — é exatamente essa disciplina que garante a exploração em ordem de distância, e por isso BFS encontra o *caminho mais curto em número de arestas* em grafos não ponderados.

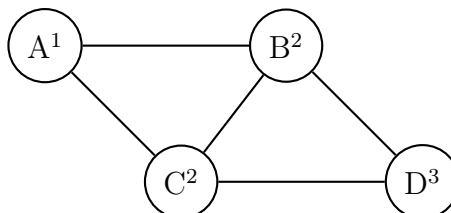


Figure 4.2: BFS a partir de A: ordem de visita = A (nível 1), B e C (nível 2), D (nível 3) — os expoentes marcam a distância em arestas.

```

1  function bfs(graph: Graph, start: string): string[] {
2      const visited = new Set<string>([start]);
3      const queue: string[] = [start];
4      const order: string[] = [];
5
6      while (queue.length > 0) {
7          const current = queue.shift()!; // idealmente uma fila real O(1)
8          order.push(current);
9          for (const neighbor of graph.neighbors(current)) {
10             if (!visited.has(neighbor)) {
11                 visited.add(neighbor);
12                 queue.push(neighbor);
13             }
14         }
15     }
16 }

```

```

15 }
16 return order; //  $O(V + E)$ 
17 }

```

Análise de Complexidade

Tempo: $O(V + E)$ — cada vértice entra na fila uma única vez, e cada aresta é examinada no máximo duas vezes (uma por extremidade). Espaço: $O(V)$ para a fila e o conjunto de visitados.

4.3 Busca em Profundidade (DFS)

DFS (*Depth-First Search*) explora o mais fundo possível por um caminho antes de retroceder (*backtrack*). Usa uma **pilha** — explícita ou implícita via recursão (que é, no fundo, a pilha de chamadas da linguagem). DFS é a base para detecção de ciclos, ordenação topológica e componentes conexos.

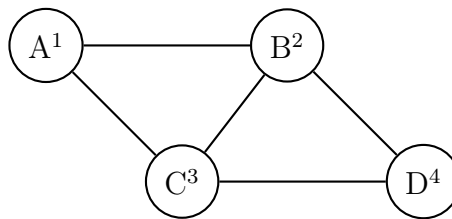


Figure 4.3: DFS a partir de A: desce por B, depois D (vizinho não visitado de B), depois C (vizinho não visitado de D) — mergulha fundo antes de explorar em largura.

```

1 function dfs(graph: Graph, start: string): string[] {
2   const visited = new Set<string>();
3   const order: string[] = [];
4
5   function visit(v: string): void {
6     visited.add(v);
7     order.push(v);
8     for (const neighbor of graph.neighbors(v)) {
9       if (!visited.has(neighbor)) visit(neighbor); // recursão = pilha
10      implícita
11    }
12  }
13
14  visit(start);
15  return order; //  $O(V + E)$ 
16 }
17
18 // Versão iterativa com pilha explícita - evita estouro de pilha
19 // em grafos muito profundos.
20 function dfsIterative(graph: Graph, start: string): string[] {

```

```

20  const visited = new Set<string>([start]);
21  const stack = [start];
22  const order: string[] = [];
23
24  while (stack.length > 0) {
25      const current = stack.pop()!;
26      order.push(current);
27      for (const neighbor of graph.neighbors(current)) {
28          if (!visited.has(neighbor)) {
29              visited.add(neighbor);
30              stack.push(neighbor);
31          }
32      }
33  }
34  return order;
35  }

```

Análise de Complexidade

Tempo: $O(V + E)$, idêntico ao BFS — a diferença está na *ordem* de exploração, não na complexidade assintótica. Espaço: $O(V)$ no pior caso (grafo em forma de caminho, pilha de recursão do tamanho de V).

4.4 Algoritmo de Dijkstra

Quando as arestas têm **pesos** (custos, distâncias, tempos), BFS deixa de encontrar o caminho mais curto — ele conta arestas, não peso acumulado. O **algoritmo de Dijkstra** resolve o caminho mais curto de um vértice de origem a todos os outros em grafos com pesos *não-negativos*, usando uma fila de prioridade (heap — Capítulo 6) para sempre expandir o vértice com menor distância acumulada conhecida.

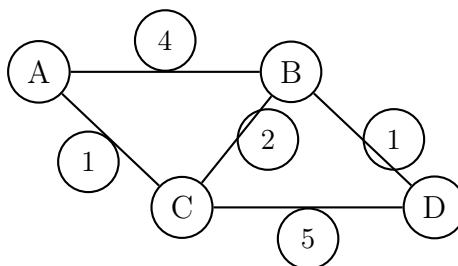


Figure 4.4: Grafo ponderado: o caminho mais curto de A a D não é A-B-D (peso 5) nem A-C-D (peso 6), mas A-C-B-D (peso $1 + 2 + 1 = 4$) — Dijkstra encontra isso explorando sempre o vértice de menor distância acumulada.

```

1  interface WeightedEdge { to: string; weight: number; }
2
3  class WeightedGraph {

```

```

4   private adjacency = new Map<string, WeightedEdge[]>();
5
6   addEdge(u: string, v: string, weight: number): void {
7       if (!this.adjacency.has(u)) this.adjacency.set(u, []);
8       if (!this.adjacency.has(v)) this.adjacency.set(v, []);
9       this.adjacency.get(u)!.push({ to: v, weight });
10      this.adjacency.get(v)!.push({ to: u, weight });
11  }
12
13  neighbors(v: string): WeightedEdge[] {
14      return this.adjacency.get(v) ?? [];
15  }
16  }
17
18  function dijkstra(graph: WeightedGraph, start: string): Map<string, number> {
19      const dist = new Map<string, number>([[start, 0]]);
20      const visited = new Set<string>();
21      // MinHeap<T> é implementado no Capítulo 6 (Heaps e Filas de Prioridade)
22      const pq = new MinHeap<{ node: string; dist: number }>((a, b) => a.dist -
23      b.dist);
24      pq.insert({ node: start, dist: 0 });
25
26      while (!pq.isEmpty()) {
27          const { node: current, dist: currentDist } = pq.extractMin();
28          if (visited.has(current)) continue;
29          visited.add(current);
30
31          for (const edge of graph.neighbors(current)) {
32              const candidate = currentDist + edge.weight;
33              if (candidate < (dist.get(edge.to) ?? Infinity)) {
34                  dist.set(edge.to, candidate);
35                  pq.insert({ node: edge.to, dist: candidate });
36              }
37          }
38      }
39      return dist; //  $O((V + E) \log V)$  com heap binário

```

Análise de Complexidade

Com fila de prioridade implementada como heap binário: $O((V + E) \log V)$. Com array simples para achar o mínimo: $O(V^2)$ — pior para grafos esparsos, competitivo apenas em grafos muito densos. Requer pesos não-negativos; para grafos com arestas negativas, use Bellman-Ford ($O(VE)$).

Chapter 5

Algoritmos de Ordenação

5.1 Bubble Sort

Bubble sort percorre o array repetidamente, trocando pares adjacentes fora de ordem. A cada passada completa, o maior elemento não posicionado “borbulha” até sua posição final. É didático, mas raramente usado em produção por sua ineficiência em n^2 .



passada 1: compara e troca adjacentes maior valor “borbulha” à direita

Figure 5.1: Uma passada de bubble sort: cada troca adjacente empurra o maior elemento visto até agora uma posição à direita.

```
1 function bubbleSort(arr: number[]): number[] {
2   const a = [...arr];
3   for (let i = 0; i < a.length - 1; i++) {
4     let swapped = false;
5     for (let j = 0; j < a.length - 1 - i; j++) {
6       if (a[j] > a[j + 1]) {
7         [a[j], a[j + 1]] = [a[j + 1], a[j]];
8         swapped = true;
9       }
10    }
11    if (!swapped) break; // já ordenado - otimização de melhor caso
12  }
13  return a;
14 }
```

Análise de Complexidade

Melhor: $O(n)$ (array já ordenado, com a otimização de **swapped**). Médio/Pior: $O(n^2)$. Espaço: $O(1)$ (in-place). Estável: sim (não troca elementos iguais de posição)

relativa).

5.2 Selection Sort

Selection sort divide o array em uma parte ordenada (esquerda) e uma não ordenada (direita). A cada iteração, encontra o menor elemento da parte não ordenada e o troca para o início dela.

```
1 function selectionSort(arr: number[]): number[] {
2   const a = [...arr];
3   for (let i = 0; i < a.length - 1; i++) {
4     let minIndex = i;
5     for (let j = i + 1; j < a.length; j++) {
6       if (a[j] < a[minIndex]) minIndex = j;
7     }
8     if (minIndex !== i) [a[i], a[minIndex]] = [a[minIndex], a[i]];
9   }
10  return a;
11 }
```

Análise de Complexidade

Melhor/Médio/Pior: sempre $O(n^2)$ — mesmo com array já ordenado, o algoritmo ainda varre toda a parte não ordenada em busca do mínimo. Espaço: $O(1)$. Vantagem sobre bubble sort: no máximo $n - 1$ trocas (útil quando escrita é cara, ex.: memória flash).

5.3 Insertion Sort

Insertion sort constrói a saída ordenada um elemento por vez, inserindo cada novo elemento na posição correta dentro da parte já ordenada — como organizar cartas de baralho na mão. É o algoritmo mais rápido na prática para arrays pequenos ou quase ordenados.

```
1 function insertionSort(arr: number[]): number[] {
2   const a = [...arr];
3   for (let i = 1; i < a.length; i++) {
4     const key = a[i];
5     let j = i - 1;
6     while (j >= 0 && a[j] > key) {
7       a[j + 1] = a[j];
8       j--;
9     }
10  }
```

```

10     a[j + 1] = key; //  $O(n)$  no total,  $O(1)$  amortizado por posição já ordenada
11 }
12 return a;
13 }

```

Análise de Complexidade

Melhor: $O(n)$ (array já ordenado). Médio/Pior: $O(n^2)$. Espaço: $O(1)$. Estável: sim. É o algoritmo escolhido internamente por **Timsort** (usado em Python e no V8 do Node.js) para sub-arrays pequenos, por ter constantes menores que $O(n \log n)$ em n pequeno.

5.4 Merge Sort

Merge sort é o primeiro algoritmo de *divisão e conquista* deste livro: divide o array ao meio recursivamente até sub-arrays de tamanho 1 (trivialmente ordenados), depois combina (*merge*) pares ordenados em sub-arrays maiores ordenados.

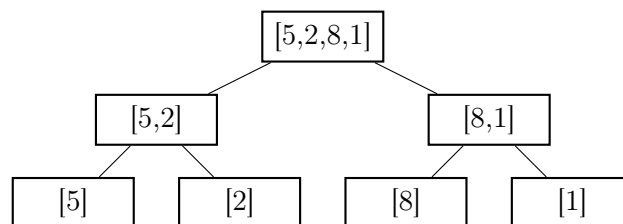


Figure 5.2: Árvore de recursão do merge sort: $\log n$ níveis de divisão, cada nível fazendo trabalho total $O(n)$ de merge $\Rightarrow O(n \log n)$.

```

1  function mergeSort(arr: number[]): number[] {
2      if (arr.length <= 1) return arr; // caso base
3
4      const mid = Math.floor(arr.length / 2);
5      const left = mergeSort(arr.slice(0, mid));
6      const right = mergeSort(arr.slice(mid));
7
8      return merge(left, right);
9  }
10
11 function merge(left: number[], right: number[]): number[] {
12     const result: number[] = [];
13     let i = 0, j = 0;
14     while (i < left.length && j < right.length) {
15         result.push(left[i] <= right[j] ? left[i++] : right[j++]);
16     }
17     return result.concat(left.slice(i), right.slice(j)); //  $O(n)$  por nível
18 }

```

Análise de Complexidade

Melhor/Médio/Pior: sempre $\Theta(n \log n)$ — não depende da entrada. Espaço: $O(n)$ extra para os arrays de merge (não é in-place). Estável: sim. Boa escolha quando estabilidade e garantia de pior caso importam mais que uso de memória (ex.: ordenar registros de banco de dados por múltiplas chaves).

5.5 Quick Sort

Quick sort também divide e conquista, mas de forma diferente: escolhe um *pivô*, particiona o array de modo que tudo menor que o pivô fique à esquerda e tudo maior à direita, e recursivamente ordena as duas partições. Ao contrário do merge sort, o trabalho pesado é na *divisão* (particionamento), não na combinação.

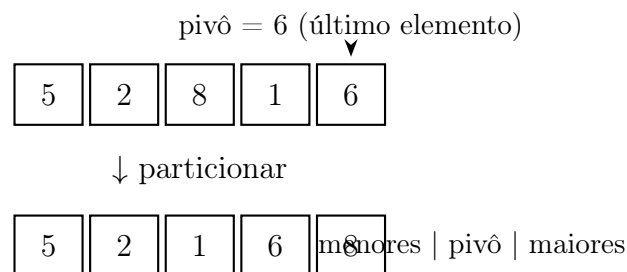


Figure 5.3: Particionamento (esquema de Lomuto): tudo < 6 vai para a esquerda do pivô, tudo > 6 para a direita — o pivô já está em sua posição final.

```

1  function quickSort(arr: number[], low = 0, high = arr.length - 1): number[] {
2    if (low < high) {
3      const pivotIndex = partition(arr, low, high);
4      quickSort(arr, low, pivotIndex - 1);
5      quickSort(arr, pivotIndex + 1, high);
6    }
7    return arr; // in-place
8  }
9
10 function partition(arr: number[], low: number, high: number): number {
11   const pivot = arr[high]; // esquema de Lomuto
12   let i = low - 1;
13   for (let j = low; j < high; j++) {
14     if (arr[j] < pivot) {
15       i++;
16       [arr[i], arr[j]] = [arr[j], arr[i]];
17     }
18   }
19   [arr[i + 1], arr[high]] = [arr[high], arr[i + 1]];
20   return i + 1; // posição final do pivô
21 }

```

Análise de Complexidade

Melhor/Médio: $\Theta(n \log n)$. Pior: $O(n^2)$ — ocorre quando o pivô escolhido é sempre o menor ou maior elemento (ex.: array já ordenado com pivô = último elemento), degenerando a partição em n fatias de tamanho 1. Mitigação comum: escolher pivô aleatório ou mediana de três. Espaço: $O(\log n)$ para a pilha de recursão. Não-estável. Na prática, geralmente mais rápido que merge sort por ser in-place e ter melhores constantes de cache.

5.6 Heap Sort

Heap sort usa a estrutura de heap binário (detalhada no Capítulo 6) para ordenar: constrói um max-heap a partir do array e repetidamente extrai o máximo, colocando-o no final do array.

```

1  function heapSort(arr: number[]): number[] {
2      const a = [...arr];
3      const n = a.length;
4
5      for (let i = Math.floor(n / 2) - 1; i >= 0; i--) heapify(a, n, i);
6
7      for (let end = n - 1; end > 0; end--) {
8          [a[0], a[end]] = [a[end], a[0]]; // move máximo atual para o fim
9          heapify(a, end, 0); // restaura propriedade de heap no restante
10     }
11     return a;
12 }
13
14 function heapify(a: number[], size: number, root: number): void {
15     let largest = root;
16     const left = 2 * root + 1;
17     const right = 2 * root + 2;
18
19     if (left < size && a[left] > a[largest]) largest = left;
20     if (right < size && a[right] > a[largest]) largest = right;
21
22     if (largest !== root) {
23         [a[root], a[largest]] = [a[largest], a[root]];
24         heapify(a, size, largest); //  $O(\log n)$ 
25     }
26 }
```

Análise de Complexidade

Melhor/Médio/Pior: sempre $\Theta(n \log n)$ — garantia forte, sem casos patológicos como quick sort. Espaço: $O(1)$ (in-place). Não-estável. Trade-off: constantes piores que quick sort na prática (menos amigável ao cache), mas com garantia de

pior caso que quick sort não oferece.

5.7 Comparativo dos Algoritmos de Ordenação

Tabela Comparativa

Algoritmo	Melhor	Médio	Pior	Espaço	Estável
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Sim
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	Não
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Sim
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Sim
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	Não
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	Não

Chapter 6

Tabelas Hash e Dicionários

6.1 Hashing e Resolução de Colisões

Uma **tabela hash** mapeia chaves a valores usando uma **função hash** que converte a chave em um índice de array. Isso dá acesso, inserção e remoção em $O(1)$ *no caso médio* — dramaticamente mais rápido que buscar em uma lista, sem exigir ordem como uma árvore. O preço é que duas chaves diferentes podem colidir no mesmo índice (*colisão*), exigindo uma estratégia de resolução.

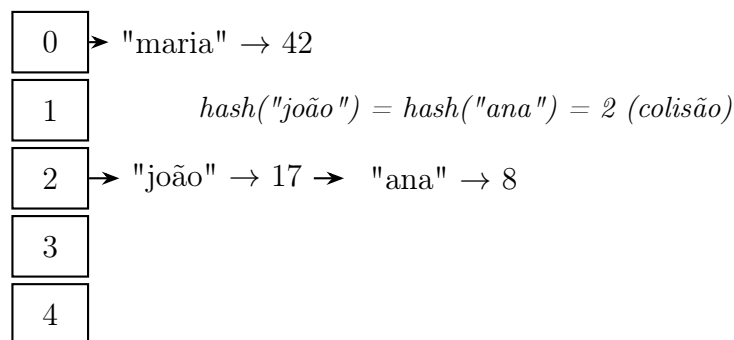


Figure 6.1: Encadeamento separado (*separate chaining*): cada posição do array guarda uma lista ligada de pares chave-valor cujas chaves colidiram no mesmo índice.

```
1 class HashMap<K extends string | number, V> {
2     private buckets: Array<[K, V] []>;
3     private size = 0;
4
5     constructor(private capacity = 16) {
6         this.buckets = Array.from({ length: capacity }, () => []);
7     }
8
9     private hash(key: K): number {
10         const str = String(key);
11         let h = 0;
12         for (let i = 0; i < str.length; i++) {
```

```

13     h = (h * 31 + str.charCodeAt(i)) >>> 0; // polinomial, evita overflow
14 }
15 return h % this.buckets.length; // O(tamanho da chave)
16 }
17
18 set(key: K, value: V): void {
19     if (this.size / this.capacity > 0.75) this.resize(); // mantém fator de
20     carga baixo
21     const bucket = this.buckets[this.hash(key)];
22     const existing = bucket.find([k] => k === key);
23     if (existing) { existing[1] = value; return; }
24     bucket.push([key, value]);
25     this.size++;
26 }
27
28 get(key: K): V | undefined {
29     const bucket = this.buckets[this.hash(key)];
30     return bucket.find([k] => k === key)?.[1]; // O(1) médio
31 }
32
33 delete(key: K): boolean {
34     const bucket = this.buckets[this.hash(key)];
35     const index = bucket.findIndex([k] => k === key);
36     if (index === -1) return false;
37     bucket.splice(index, 1);
38     this.size--;
39     return true;
40 }
41
42 private resize(): void {
43     const old = this.buckets;
44     this.capacity *= 2;
45     this.buckets = Array.from({ length: this.capacity }, () => []);
46     this.size = 0;
47     for (const bucket of old) for (const [k, v] of bucket) this.set(k, v);
48 }

```

Análise de Complexidade

Operação	Caso médio	Pior caso
Acesso/Busca	$O(1)$	$O(n)$ (todas as chaves colidem)
Inserção	$O(1)$ amortizado	$O(n)$
Remoção	$O(1)$	$O(n)$

O pior caso $O(n)$ ocorre com uma função hash ruim ou ataque proposital (*hash flooding*); na prática, com boas funções hash e fator de carga controlado (< 0.75 , gatilho de *resize*), o caso médio $O(1)$ domina. Alternativa a encadeamento: *open addressing* (linear/quadratic probing), que evita listas ligadas mas é mais sensível a fator de carga alto.

Chapter 7

Heaps e Filas de Prioridade

7.1 Binary Heap

Um **heap binário** é uma árvore binária quase completa (todos os níveis cheios exceto possivelmente o último, preenchido da esquerda para a direita) que satisfaz a *propriedade de heap*: em um **min-heap**, todo nó é $\leq$ seus filhos (o mínimo está sempre na raiz); em um **max-heap**, todo nó é $\geq$ seus filhos. Essa estrutura é representada de forma compacta em um **array**, sem precisar de ponteiros: para um nó no índice i , o pai está em $\lfloor (i-1)/2 \rfloor$, o filho esquerdo em $2i+1$, o direito em $2i+2$.

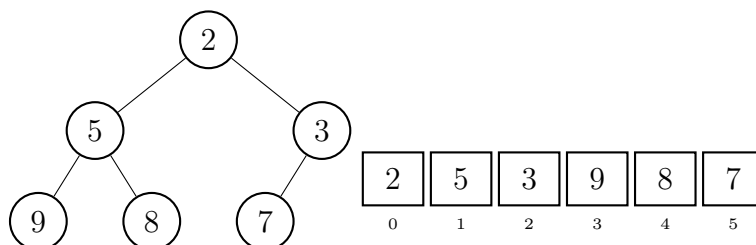


Figure 7.1: Min-heap como árvore (topo) e sua representação equivalente como array (baixo) — filho esquerdo/direito do índice i são $2i+1/2i+2$, sem necessidade de ponteiros.

```
1 class MinHeap<T> {
2     private items: T[] = [];
3     constructor(private compare: (a: T, b: T) => number) {}
4
5     isEmpty(): boolean {
6         return this.items.length === 0;
7     }
8
9     private parent(i: number) { return Math.floor((i - 1) / 2); }
10    private left(i: number) { return 2 * i + 1; }
11    private right(i: number) { return 2 * i + 2; }
12    private swap(i: number, j: number) {
13        [this.items[i], this.items[j]] = [this.items[j], this.items[i]];
14    }
```

```
15
16 insert(value: T): void {
17     this.items.push(value);
18     this.bubbleUp(this.items.length - 1); // O(log n)
19 }
20
21 private bubbleUp(index: number): void {
22     while (index > 0) {
23         const parentIdx = this.parent(index);
24         if (this.compare(this.items[index], this.items[parentIdx]) >= 0) break;
25         this.swap(index, parentIdx);
26         index = parentIdx;
27     }
28 }
29
30 extractMin(): T {
31     const min = this.items[0];
32     const last = this.items.pop()!;
33     if (this.items.length > 0) {
34         this.items[0] = last;
35         this.bubbleDown(0); // O(log n)
36     }
37     return min;
38 }
39
40 private bubbleDown(index: number): void {
41     let smallest = index;
42     const l = this.left(index), r = this.right(index);
43     if (l < this.items.length && this.compare(this.items[l],
44         this.items[smallest]) < 0) {
45         smallest = l;
46     }
47     if (r < this.items.length && this.compare(this.items[r],
48         this.items[smallest]) < 0) {
49         smallest = r;
50     }
51     if (smallest !== index) {
52         this.swap(index, smallest);
53         this.bubbleDown(smallest);
54     }
55 }
```

Análise de Complexidade

Operação	Complexidade
Ver mínimo (peek)	$O(1)$
Inserir	$O(\log n)$
Extrair mínimo	$O(\log n)$
Construir heap a partir de array (heapify)	$O(n)$

Heaps são a estrutura padrão para **filas de prioridade**, usadas em Dijkstra (Capítulo 4), escalonadores de sistemas operacionais e algoritmos de compressão (Huffman). **heapify** completo é $O(n)$, não $O(n \log n)$ como se poderia supor — a soma das alturas de todos os nós converge para $O(n)$.

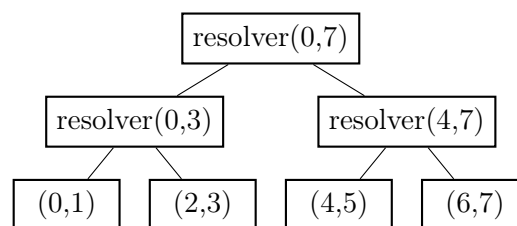
Chapter 8

Técnicas de Projeto de Algoritmos

Esta seção final amarra os capítulos anteriores mostrando o mesmo problema — **soma máxima de subarray contíguo** — resolvido com três técnicas diferentes, tornando os trade-offs concretos e comparáveis.

8.1 Divisão e Conquista

Divisão e conquista resolve um problema dividindo-o em subproblemas menores do mesmo tipo, resolvendo-os recursivamente, e combinando as soluções. Merge sort e quick sort (Capítulo 5) já são exemplos centrais. Aqui aplicamos a técnica à soma máxima de subarray: divida o array ao meio; a resposta está inteiramente à esquerda, inteiramente à direita, ou cruza o meio.



+ combinar: melhor cruzando o meio, em cada nível

Figure 8.1: Árvore de recursão: $O(\log n)$ níveis, $O(n)$ trabalho de combinação por nível $\Rightarrow O(n \log n)$ no total.

```
1 function maxSubarrayDC(arr: number[], low = 0, high = arr.length - 1): number {
2   if (low === high) return arr[low]; // caso base
3
4   const mid = Math.floor((low + high) / 2);
5   const leftMax = maxSubarrayDC(arr, low, mid);
6   const rightMax = maxSubarrayDC(arr, mid + 1, high);
```

```

7  const crossMax = maxCrossingSum(arr, low, mid, high);
8
9  return Math.max(leftMax, rightMax, crossMax);
10 }
11
12 function maxCrossingSum(arr: number[], low: number, mid: number, high: number):
number {
13   let leftSum = -Infinity, sum = 0;
14   for (let i = mid; i >= low; i--) {
15     sum += arr[i];
16     leftSum = Math.max(leftSum, sum);
17   }
18   let rightSum = -Infinity;
19   sum = 0;
20   for (let i = mid + 1; i <= high; i++) {
21     sum += arr[i];
22     rightSum = Math.max(rightSum, sum);
23   }
24   return leftSum + rightSum; // O(n) por nível de recursão
25 }

```

Análise de Complexidade

Tempo: $T(n) = 2T(n/2) + O(n) \Rightarrow O(n \log n)$ pelo Teorema Mestre. Espaço: $O(\log n)$ de pilha de recursão. Serve de base conceitual para entender programação dinâmica: quando os subproblemas se sobrepõem (o que não acontece aqui), memoizar essas chamadas leva à próxima técnica.

8.2 Programação Dinâmica

Programação dinâmica (PD) se aplica quando um problema tem **subproblemas sobrepostos** e **subestrutura ótima** — a solução ótima é composta de soluções ótimas dos subproblemas. Diferente de divisão e conquista pura, PD *armazena* (memoiza) resultados de subproblemas para nunca recalculá-los. O exemplo clássico é Fibonacci: a versão recursiva ingênua recalcula os mesmos valores exponencialmente; memoização reduz isso a tempo linear.

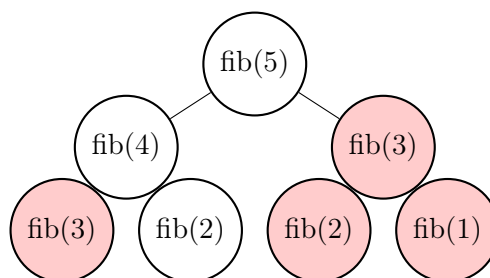


Figure 8.2: Recursão ingênua de Fibonacci recalcula `fib(3)` e `fib(2)` (destacados) múltiplas vezes — exatamente esse recálculo é o que a memoização elimina.

```

1  // Top-down: memoização sobre a recursão natural
2  function fibMemo(n: number, memo = new Map<number, number>()): number {
3      if (n <= 1) return n;
4      if (memo.has(n)) return memo.get(n)!; // O(1) - evita recomputar
5      const result = fibMemo(n - 1, memo) + fibMemo(n - 2, memo);
6      memo.set(n, result);
7      return result; // O(n) no total, O(n) subproblemas únicos
8  }
9
10 // Bottom-up: tabulação, sem recursão nem risco de estouro de pilha
11 function fibTabulation(n: number): number {
12     if (n <= 1) return n;
13     const table = new Array(n + 1).fill(0);
14     table[1] = 1;
15     for (let i = 2; i <= n; i++) table[i] = table[i - 1] + table[i - 2];
16     return table[n]; // O(n) tempo, O(n) espaço
17 }
18
19 // Aplicação clássica de PD: mochila 0/1 (0/1 knapsack)
20 function knapsack(weights: number[], values: number[], capacity: number):
21 number {
22     const n = weights.length;
23     const dp: number[][] = Array.from({ length: n + 1 }, () => new Array(capacity
24 + 1).fill(0));
25
26     for (let i = 1; i <= n; i++) {
27         for (let w = 0; w <= capacity; w++) {
28             dp[i][w] = dp[i - 1][w]; // não inclui o item i
29             if (weights[i - 1] <= w) {
30                 dp[i][w] = Math.max(dp[i][w], dp[i - 1][w - weights[i - 1]] + values[i
31 - 1]);
32             }
33         }
34     }
35     return dp[n][capacity]; // O(n * capacity)
36 }

```

Análise de Complexidade

Fibonacci: recursão ingênua $O(2^n)$; com memoização, $O(n)$ tempo e $O(n)$ espaço. Mochila 0/1: $O(n \times W)$ tempo e espaço, onde W é a capacidade — pseudo-polinomial (depende do *valor* de W , não apenas do tamanho da entrada). Regra prática: se a recursão natural recalcula os mesmos argumentos, PD quase certamente se aplica.

8.3 Algoritmos Gulosos (Greedy)

Algoritmos **gulosos** constroem a solução passo a passo, escolhendo a cada etapa a opção que parece melhor *naquele momento*, sem reconsiderar decisões passadas. Funcionam apenas quando o problema tem a **propriedade de escolha gulosa** — uma sequência de escolhas localmente ótimas leva a uma solução globalmente ótima. Quando essa propriedade não vale, greedy dá uma resposta rápida, mas não necessariamente correta.

25	10	5	1
----	----	---	---

trocar 63 centavos: $2 \times 25 + 1 \times 10 + 3 \times 1 = 6$ moedas

Figure 8.3: Troco guloso: sempre escolhe a maior moeda que cabe no valor restante. Funciona para o sistema monetário usual, mas falha para sistemas de moedas arbitrários (ex.: $[1, 3, 4]$ para trocar 6 — guloso dá $4+1+1=3$ moedas, ótimo real é $3+3=2$ moedas).

```

1  // Troco guloso - correto apenas para sistemas "canônicos" de moedas
2  function coinChangeGreedy(coins: number[], amount: number): number[] {
3      const sorted = [...coins].sort((a, b) => b - a);
4      const result: number[] = [];
5      let remaining = amount;
6
7      for (const coin of sorted) {
8          while (remaining >= coin) {
9              result.push(coin);
10             remaining -= coin; // decisão local, nunca reconsiderada
11         }
12     }
13     return remaining === 0 ? result : []; // pode falhar em sistemas
14     não-canônicos
15 }
16
17 // Seleção de atividades: escolher o máximo de atividades sem sobreposição.
18 // A escolha gulosa correta aqui é ordenar por horário de TÉRMINO, não de
19 // duração.
20 interface Activity { start: number; end: number; }
21
22 function activitySelection(activities: Activity[]): Activity[] {
23     const sorted = [...activities].sort((a, b) => a.end - b.end);
24     const selected: Activity[] = [sorted[0]];
25     let lastEnd = sorted[0].end;
26
27     for (let i = 1; i < sorted.length; i++) {
28         if (sorted[i].start >= lastEnd) {
29             selected.push(sorted[i]); // sempre aceita a próxima compatível
30             lastEnd = sorted[i].end;
31         }
32     }
33     return selected; // O(n log n), dominado pela ordenação

```

32

}

Análise de Complexidade

Troco guloso: $O(n \log n)$ para ordenar as moedas, mas **correção não garantida** — só é ótimo para sistemas de moedas canônicos (como o real ou o dólar). Seleção de atividades: $O(n \log n)$, e aqui a escolha gulosa (ordenar por término mais cedo) é *comprovadamente* ótima — a diferença entre os dois exemplos é exatamente o ponto: greedy exige prova, não só intuição.

Appendix A

Referências e Leitura Adicional

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., Stein, C. *Introduction to Algorithms* (3rd ed.), MIT Press — a referência canônica para todos os temas deste livro, incluindo as provas formais de complexidade omitidas aqui por brevidade.
- Sedgwick, R., Wayne, K. *Algorithms* (4th ed.), Addison-Wesley — abordagem mais prática, com implementações de referência em Java.
- Skiena, S. S. *The Algorithm Design Manual* (2nd ed.), Springer — forte em heurísticas de quando aplicar cada técnica na prática.
- MDN Web Docs, *TypeScript Handbook* (<https://www.typescriptlang.org/docs/>) — documentação oficial da linguagem usada nos exemplos de código deste livro.
- PostgreSQL Documentation, *Index Types* (<https://www.postgresql.org/docs/current/indexes-types.html>) — uso real de B-trees como índice padrão em bancos relacionais.
- Cormen et al., capítulo sobre *Red-Black Trees* — para a prova completa das cinco invariantes e do algoritmo de remoção, que este livro resume de forma introdutória.

Appendix B

Glossário

Algoritmo guloso (greedy): técnica que escolhe, a cada passo, a opção localmente ótima, sem retroceder.

BFS (Busca em Largura): percorre um grafo por camadas de distância, usando uma fila.

Big-O: notação que descreve o limite superior assintótico do crescimento de uma função.

DFS (Busca em Profundidade): percorre um grafo mergulhando fundo em um caminho antes de retroceder, usando uma pilha (explícita ou por recursão).

Fator de balanceamento: diferença de altura entre as subárvores esquerda e direita de um nó, usada por árvores AVL.

Fator de carga: razão entre número de elementos e número de posições (buckets) em uma tabela hash; controla quando fazer resize.

Heap: árvore binária quase completa que satisfaz a propriedade de heap (min ou max), representável eficientemente como array.

Memoização: técnica de armazenar resultados de subproblemas já calculados para evitar recomputação, base da programação dinâmica top-down.

Pivô: elemento escolhido em quick sort para particionar o array em menores e maiores.

Rotação: operação estrutural que reorganiza nós de uma árvore para restaurar balanceamento (AVL) ou invariantes de coloração (rubro-negra), preservando a ordem de busca.

Subestrutura ótima: propriedade de um problema onde a solução ótima é composta de soluções ótimas de seus subproblemas — pré-requisito para programação dinâmica.

Index

- algoritmo guloso, 39
 - definição, 42
- array, 6
- AVL, 13
 - fator de balanceamento, 42
 - rotação, 42
- B-tree, 17
- BFS, 21
 - definição, 42
- Big-O, 3
 - definição, 42
- Big-Omega, 3
- Big-Theta, 4
- BST, 12
- bubble sort, 25
- complexidade
 - motivação, 3
- DFS, 22
 - definição, 42
- Dijkstra, 23
- divisão e conquista, 36
- fila, 10
- fila de prioridade, 33
- grafo, 20
- hash, 31
- heap, 33
 - definição, 42
- heap sort, 29
- insertion sort, 26
- lista ligada, 7
- merge sort, 27
- pilha, 9
- programação dinâmica, 37
 - memoização, 42
 - subestrutura ótima, 42
- quick sort, 28
 - pivô, 42
- selection sort, 26
- tabela hash, 31
 - fator de carga, 42
- árvore binária, 12
- árvore rubro-negra, 15
 - rotação, 42